Task 1: Guided Activity, Student result analysis

```
import Foundation
struct Student {
   let name: String
   let marks: Int
   let dept: String
}

let students = [
   Student(name: "Asha", marks: 82, dept: "IT"),
   Student(name: "Ravi", marks: 35, dept: "CSE"),
   Student(name: "Meena", marks: 91, dept: "IT"),
   Student(name: "Kiran", marks: 56, dept: "ECE"),
   Student(name: "Divya", marks: 67, dept: "CSE"),
   Student(name: "Arjun", marks: 40, dept: "IT")
]
let passed = students.filter { student in
   student.marks >= 40
}
print("Passed Students:", passed.map { $0.name })

let names = students.map { $0.name }
print("All Names:", names)

let total = students.map { $0.marks }.reduce(0, +)
let average = Double(total) / Double(students.count)
print("Class Average:", average)

let ranked = students.sorted { $0.marks > $1.marks }

print("Rankings:")
```

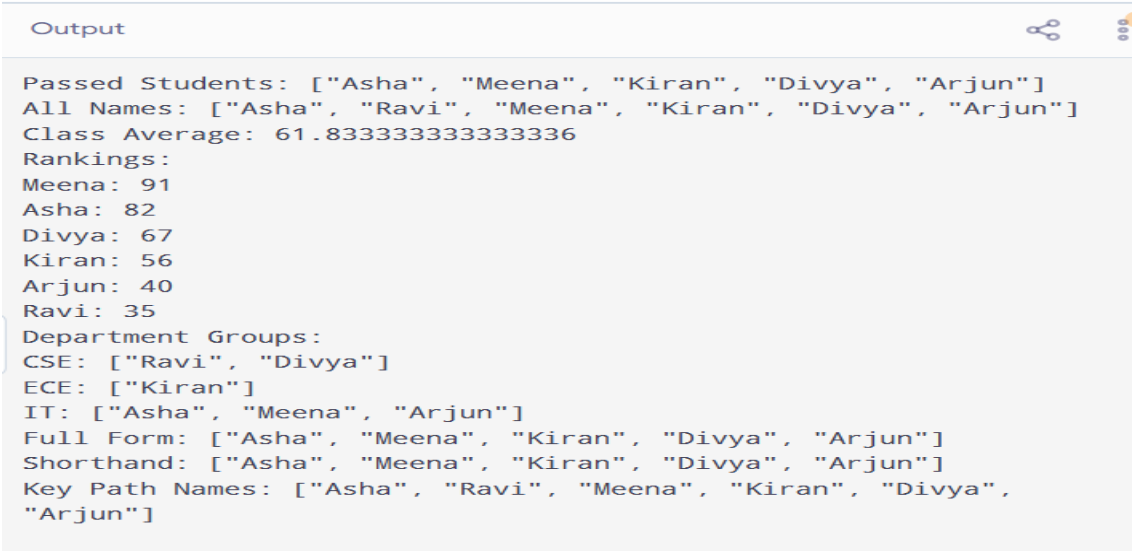
```
for student in ranked {
    print("\(student.name): \(student.marks)")
}

let grouped = Dictionary(grouping: students, by: { $0.dept })

print("Department Groups:")
for dept in grouped.keys.sorted() {
    let departmentNames = grouped[dept]!.map { $0.name }
    print("\(dept): \(departmentNames)")
}

let passedFull = students.filter { student in
    return student.marks >= 40
}
print("Full Form:", passedFull.map { $0.name })
let passedShort = students.filter { $0.marks >= 40 }
print("Shorthand:", passedShort.map { $0.name })
let namesUsingKeyPath = students.map(\.name)
print("Key Path Names:", namesUsingKeyPath)
```

output:



Task 2: Practice Task, Write your own higher-order functions

```
import Foundation


func myFilter(_ items: [Int], _ predicate: (Int) -> Bool) -> [Int] {
   var result: [Int] = []


   for item in items {
      if predicate(item) {
         result.append(item)
      }
   }


   return result
}
func applyTwice(_ value: Int, _ f: (Int) -> Int) -> Int {
   return f(f(value))
}
class Button {
   var handlers: [() -> Void] = []


   func onTap(_ handler: @escaping () -> Void) {
      handlers.append(handler)
   }


   func tap() {
      for handler in handlers {
         handler()
      }
   }
}
let numbers = [1, 2, 3, 4, 5, 6]
```

```
let evenNumbers = myFilter(numbers) { $0 % 2 == 0 }
print("Even Numbers:", evenNumbers)
let result = applyTwice(3) { $0 * 2 }
print("Apply Twice:", result)
let button = Button()

button.onTap {
  print("Button tapped!")
}

button.onTap {
  print("Second handler executed!")
}
button.tap()
```

output:

Output

```
Even Numbers: [2, 4, 6]
Apply Twice: 12
Button tapped!
Second handler executed!
```

Task 3: Predict Task, Closure capture

```
func makeCounter() -> () -> Int {
  var count = 0
  return {
    count += 1
    return count
  }
}

let a = makeCounter()
```

let b = makeCounter()

print(a(), a(), b(), a())

output:

